Practical No. 8

C Program: Banker's Algorithm (Deadlock Avoidance)

1. AIM

To write a C program to implement the Banker's Algorithm: calculate the Need matrix, check whether the system is in a safe state, and display the safe sequence.

2. REQUIREMENTS

- Operating System: Ubuntu / Linux (or Windows)
- Language: C, Compiler: GCC
- Text editor: Nano / Vim / VS Code
- Hardware: computer, keyboard, monitor

3. SHORT THEORY

- Deadlock: a situation where processes wait forever for resources held by each other.
- Banker's Algorithm: a deadlock AVOIDANCE algorithm (developed by Edsger W. Dijkstra). Before giving resources to a process, it checks whether the system will stay in a safe state. It is called Banker's because a bank never gives out cash if it cannot satisfy all customers later.
- Safe state: all processes can finish in some order without deadlock.
- Safe sequence: that order of processes. (There can be more than one.)
- Unsafe state: no safe sequence exists. It may lead to deadlock.
- Allocation matrix: resources currently held by each process.
- Max matrix: maximum resources each process may need.
- Available: resources that are free right now.
- Need matrix: resources each process still needs.

Formula	Meaning
`Need = Max - Allocation`	Calculated for every process and every resource type.
`Available = total - allocated`	Free resources at the moment.
`Condition to run process Pi`	Need of Pi <= Available (for every resource).
`After Pi finishes`	Available = Available + Allocation of Pi.

Four conditions for deadlock: mutual exclusion, hold and wait, no preemption, circular wait.

4. ALGORITHM

- 1. Start. Enter number of processes (n) and resources (m).
- 2. Enter the Allocation matrix, Max matrix and Available resources.
- 3. Calculate Need = Max - Allocation.
- 4. Find an unfinished process whose Need <= Available.
- 5. If found, assume it finishes: add its Allocation to Available, and add it to the safe sequence.
- 6. Repeat step 4-5 until all processes finish, or no such process can be found.
- 7. If all finished: print SAFE and the safe sequence. Otherwise print UNSAFE. Stop.

5. CODE (file name: banker.c)

```
#include <stdio.h>

int main()
{
    int n, m, i, j, k;
    int alloc[10][10], max[10][10];
    int need[10][10], avail[10];
    int finish[10] = {0}, safe[10];
    int count = 0, found;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available Resources:\n");
    for(j = 0; j < m; j++)
        scanf("%d", &avail[j]);

    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    printf("\nNeed Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }

    while(count < n)
    {
        found = 0;
        for(i = 0; i < n; i++)
        {
            if(finish[i] == 0)
            {
                for(j = 0; j < m; j++)
                    if(need[i][j] > avail[j])
                        break;

                if(j == m)
                {
                    for(k = 0; k < m; k++)
                        avail[k] += alloc[i][k];
                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if(found == 0)
            break;
```

```
    }

    if(count == n)
    {
        printf("\nSystem is in SAFE state.\n");
        printf("Safe Sequence: ");
        for(i = 0; i < n; i++)
            printf("P%d ", safe[i]);
    }
    else
    {
        printf("\nSystem is in UNSAFE state.");
    }
    printf("\n");
    return 0;
}
```

6. HOW TO EXECUTE (step by step)

Command / Key	What it does
`Ctrl + Alt + T`	Open the Ubuntu terminal.
`sudo apt install gcc`	Only if gcc is not installed.
`cd ~`	Go to the home folder.
`nano banker.c`	Create the file, paste the code, save (Ctrl+O, Enter, Ctrl+X).
`gcc banker.c -o banker`	Compile the program.
`./banker`	Run it. Type the numbers when asked (rows separated by Enter, values by space).

7. LINE-BY-LINE EXPLANATION

Code	Meaning
`#include <stdio.h>`	Adds the input/output library (printf, scanf).
`int n, m, i, j, k;`	n = processes, m = resource types, i, j, k = loop counters.
`int alloc[10][10], max[10][10];`	Two-dimensional arrays (rows = processes, columns = resources) for Allocation and Max.
`int need[10][10], avail[10];`	Need matrix, and Available resources (one value per resource).
`int finish[10] = {0}, safe[10];`	finish[i] = 0 means process i not finished yet (all start at 0). safe[] stores the safe sequence.
`int count = 0, found;`	count = number of processes finished so far. found = flag, did we find a runnable process in this pass.
`scanf("%d", &alloc[i][j]);`	Reads the Allocation matrix value by value using two nested loops.
`(same for max and avail)`	Reads the Max matrix and the Available resources.
`need[i][j] = max[i][j] - alloc[i][j];`	Need = Max - Allocation for each element.
`printf("%d ", need[i][j]);`	Prints the Need matrix row by row; printf("\n") ends each row.
`while(count < n)`	Keep going until all processes are finished.
`found = 0;`	At the start of each pass, assume nothing was found.

Code	Meaning
`for(i = 0; i < n; i++)`	Look at every process one by one.
`if(finish[i] == 0)`	Only check processes that are not finished.
`if(need[i][j] > avail[j]) break;`	If this process needs more than available for any resource, stop checking it.
`if(j == m)`	The inner loop ran to the end without break, so Need <= Available for ALL resources. The process can run.
`avail[k] += alloc[i][k];`	The process finishes and gives back its resources, so Available increases.
`safe[count++] = i;`	Adds process i to the safe sequence, then increases count by 1.
`finish[i] = 1; found = 1;`	Mark the process finished and note that we found one in this pass.
`if(found == 0) break;`	No process could run in a full pass, so stop. The system is unsafe.
`if(count == n)`	All processes finished, so the system is SAFE. Print the safe sequence.
`else`	Some processes could not finish, so the system is UNSAFE.

8. MEANING OF SYMBOLS

Symbol	Use
`[ ][ ]`	Two-dimensional array (matrix). alloc[1][2] = row 1, column 2.
`{0}`	Sets all elements of the array to 0.
`&`	Address-of. scanf needs it to store the value in the variable.
`%d`	Format code for an integer.
`+=`	Add and store: avail += x means avail = avail + x.
`count++`	Use count, then increase it by 1.
`break`	Immediately leaves the current loop.
`== > <`	Equal to, greater than, less than (comparison).

9. SAMPLE INPUT AND OUTPUT

5 processes (P0 to P4), 3 resources (A, B, C). Available = 3 3 2

Process data	Need = Max - Alloc
`P0: Alloc 0 1 0 Max 7 5 3`	Need 7 4 3
`P1: Alloc 2 0 0 Max 3 2 2`	Need 1 2 2
`P2: Alloc 3 0 2 Max 9 0 2`	Need 6 0 0
`P3: Alloc 2 1 1 Max 2 2 2`	Need 0 1 1
`P4: Alloc 0 0 2 Max 4 3 3`	Need 4 3 1

How the safe sequence is found (start Available = 3 3 2):

Step	New Available
P1 (need 1 2 2 <= 3 3 2)	Available = 3 3 2 + 2 0 0 = 5 3 2

Step	New Available
P3 (need 0 1 1 <= 5 3 2)	Available = 5 3 2 + 2 1 1 = 7 4 3
P4 (need 4 3 1 <= 7 4 3)	Available = 7 4 3 + 0 0 2 = 7 4 5
P0 (need 7 4 3 <= 7 4 5)	Available = 7 4 5 + 0 1 0 = 7 5 5
P2 (need 6 0 0 <= 7 5 5)	Available = 7 5 5 + 3 0 2 = 10 5 7

Output: System is in SAFE state. Safe Sequence: P1 P3 P4 P0 P2. (I compiled and ran the code to confirm this. If Available is 0 0 0, the output is UNSAFE.) Note: this program only checks safety; it does not process a new resource request.

10. CONCLUSION

Thus, the Banker's Algorithm was successfully implemented in C. The program calculates the Need matrix and checks the safety of the system. For the given input the system is in a safe state and one safe sequence is P1 -> P3 -> P4 -> P0 -> P2.

11. QUICK VIVA ANSWERS

- Banker's Algorithm: a deadlock avoidance algorithm.
- Developed by: Edsger W. Dijkstra.
- Deadlock: processes wait indefinitely for resources held by each other.
- Safe state: a state in which all processes can complete in some order.
- Need matrix: additional resources each process may still request (Need = Max - Allocation).
- Avoidance vs prevention: prevention removes one of the 4 deadlock conditions; avoidance checks safety before each allocation (Banker's).